

Doc 7.1 — Object Fundamentals & Access

JavaScript-এর সবচেয়ে important data structures-এর মধ্যে **Object** অন্যতম।

Real-world application-এ user, product, order, employee, settings, API response—প্রায় সবকিছুই object আকারে represent করা হয়।

1. Object কী?

Object হলো এমন একটি data structure যেখানে আমরা data-কে **key-value pair** আকারে store করি।

```
const user = {  
  name: "Shariar",  
  age: 31,  
  isAdmin: false  
};
```

এখানে:

Key / Property	Value
name	"Shariar"
age	31
isAdmin	false

Object-এর প্রতিটি key-কে বলা হয় **property**।

```
Object  
|  
├─ name → "Shariar"  
├─ age → 31  
└─ isAdmin → false
```

সহজভাবে মনে রাখো

Object = related data-কে key-value pair আকারে organize করার container

2. কেন Object ব্যবহার করি?

ধরো একজন user-এর information রাখতে হবে।

Object ছাড়া:

```
const userName = "Shariar";  
const userAge = 31;  
const userEmail = "shariar@example.com";  
const userIsAdmin = false;
```

অনেকগুলো আলাদা variable হয়ে গেল।

Object ব্যবহার করলে:

```
const user = {  
  name: "Shariar",  
  age: 31,  
  email: "shariar@example.com",  
  isAdmin: false  
};
```

এখন সব related information এক জায়গায়।

```
user.name;  
user.age;  
user.email;
```

এটাই object-এর মূল সুবিধা।

3. Object Literal

Object তৈরি করার সবচেয়ে common এবং recommended way হলো **object literal**।

Syntax:

```
const objectName = {  
  key: value,  
  key: value  
};
```

Example:

```
const person = {  
  name: "Rahim",  
  age: 25,  
  city: "Dhaka"  
};
```

এখানে:

```
name: "Rahim"  
age: 25  
city: "Dhaka"
```

প্রতিটি হলো একটি property।

4. Empty Object

Empty object তৈরি করা যায় এভাবে:

```
const user = {};
```

এখন object-এর মধ্যে কোনো property নেই।

পরে property add করতে পারি:

```
user.name = "Shariar";  
user.age = 31;
```

এখন:

```
console.log(user);
```

Output:

```
{  
  name: "Shariar",  
  age: 31  
}
```

5. Object-এর Property

Object-এর property হলো:

```
key → value
```

Example:

```
const product = {  
  name: "Laptop",  
  price: 80000,  
  brand: "HP"  
};
```

এখানে:

```
name → "Laptop"  
price → 80000  
brand → "HP"
```

Value যেকোনো valid JavaScript value হতে পারে।

6. Object-এ বিভিন্ন ধরনের Value

Object-এর property value হতে পারে:

String

```
const user = {  
  name: "Shariar"  
};
```

Number

```
const user = {  
  age: 31  
};
```

Boolean

```
const user = {  
  isLoggedIn: true  
};
```

Null

```
const user = {  
  middleName: null  
};
```

Array

```
const user = {  
  skills: ["JavaScript", "React", "Next.js"]  
};
```

Function

```
const user = {  
  greet() {  
    console.log("Hello");  
  }  
};
```

Another Object

```
const user = {  
  address: {  
    city: "Dhaka",  
    country: "Bangladesh"  
  }  
};
```

অর্থাৎ object-এর ভিতরে প্রায় যেকোনো JavaScript value রাখা যায়।

7. Dot Notation

Object property access করার সবচেয়ে common way হলো **dot notation**।

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
console.log(user.name);  
console.log(user.age);
```

Output:

```
Shariar  
31
```

Syntax:

```
object.property
```

Example:

```
user.name
```

এখানে:

```
user → object  
name → property
```

8. Dot Notation-এর আরও Example

```
const product = {
  title: "MacBook",
  price: 120000,
  stock: 5
};

console.log(product.title);
console.log(product.price);
console.log(product.stock);
```

Output:

```
MacBook
120000
5
```

9. Bracket Notation

Object property access করার আরেকটি গুরুত্বপূর্ণ way হলো **bracket notation**।

Syntax:

```
object["property"]
```

Example:

```
const user = {
  name: "Shariar",
  age: 31
};

console.log(user["name"]);
console.log(user["age"]);
```

Output:

Shariar

31

10. Dot Notation বনাম Bracket Notation

দুটোই একই property access করতে পারে।

```
user.name;
```

এবং:

```
user["name"];
```

দুটোর result একই।

```
const user = {  
  name: "Shariar"  
};  
  
console.log(user.name);  
console.log(user["name"]);
```

Output:

```
Shariar  
Shariar
```

11. তাহলে Bracket Notation কেন দরকার?

সবচেয়ে important কারণ হলো **dynamic property access**।

ধরো:

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const key = "name";  
  
console.log(user[key]);
```

Output:

```
Shariar
```

এখানে:

```
user[key]
```

মানে:

```
user["name"]
```

12. Dot Notation-এর একটি গুরুত্বপূর্ণ limitation

ধরো:

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const key = "name";
```

এটা কাজ করবে:

```
user[key];
```

কিন্তু এটা dynamic property access করবে না:

```
user.key;
```

কারণ:

```
user.key
```

মানে object-এর `"key"` নামের property খোঁজা।

আর:

```
user[key]
```

মানে `key` variable-এর value ব্যবহার করা।

যেহেতু:

```
key = "name"
```

তাই:

```
user[key]
```

হয়ে যায়:

```
user["name"]
```

13. Dot vs Bracket — Mental Model

```
user.name
```

মানে:

"name নামের property দাও"

আর:

```
user[key]
```

মানে:

"key variable-এর ভিতরে যে property name আছে, সেই property দাও"

14. কখন Dot Notation ব্যবহার করব?

Property name যদি normal identifier হয়:

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
user.name;  
user.age;
```

তাহলে সাধারণত dot notation সবচেয়ে clean।

15. কখন Bracket Notation ব্যবহার করব?

Dynamic key

```
const key = "email";  
  
user[key];
```

Space থাকা property

```
const user = {  
  "full name": "Shariar Mahamud"  
};  
  
console.log(user["full name"]);
```

এখানে:

```
user.full name
```

valid syntax নয়।

16. Special Character Property

Property name-এ এমন character থাকতে পারে যেটা dot notation দিয়ে সহজে access করা যায় না।

```
const user = {  
  "first-name": "Shariar"  
};  
  
console.log(user["first-name"]);
```

Bracket notation এখানে appropriate।

17. Object Property Add করা

Object তৈরি করার পর নতুন property add করা যায়।

```
const user = {  
  name: "Shariar"  
};  
  
user.age = 31;
```

এখন:

```
console.log(user);
```

Output:

```
{  
  name: "Shariar",  
  age: 31  
}
```

18. Bracket দিয়ে Property Add

```
const user = {};  
  
user["name"] = "Shariar";  
user["age"] = 31;
```

Result:

```
{  
  name: "Shariar",  
  age: 31  
}
```

19. Existing Property Update

একইভাবে existing property-এর value পরিবর্তন করা যায়।

```
const user = {  
  name: "Shariar",  
  age: 30  
};  
  
user.age = 31;
```

এখন:

```
console.log(user.age);
```

Output:

```
31
```

20. Property Add বনাম Update

JavaScript আলাদাভাবে `add()` বা `update()` method ব্যবহার করে না।

তুমি assignment করলেই:

```
object.key = value;
```

Property না থাকলে:

নতুন property তৈরি হবে।

Property থাকলে:

existing value update হবে।

Example:

```
const user = {  
  name: "Shariar"  
};  
  
user.age = 31;           // Add  
user.name = "Duke";     // Update
```

21. Property Delete করা

Object থেকে property remove করতে:

```
delete object.property;
```

Example:

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
delete user.age;
```

এখন:

```
console.log(user);
```

Output:

```
{  
  name: "Shariar"  
}
```

22. Bracket দিয়ে Delete

```
delete user["name"];
```

এটাও valid।

23. `delete` কী Return করে?

সাধারণত successful deletion হলে:

```
true
```

Example:

```
const user = {  
  name: "Shariar"  
};  
  
console.log(delete user.name);
```

Output:

```
true
```

24. Object Property না থাকলে কী হয়?

```
const user = {  
  name: "Shariar"  
};  
  
console.log(user.age);
```

Output:

```
undefined
```

JavaScript error দেয় না শুধু missing property access করার জন্য।

25. Missing Property vs Property with `undefined`

দুটো দেখতে একই result দিতে পারে:

```
const user = {  
  name: "Shariar"  
};  
  
console.log(user.age);
```

এবং:

```
const user = {  
  name: "Shariar",  
  age: undefined  
};  
  
console.log(user.age);
```

দুটোতেই:

```
undefined
```

কিন্তু conceptually এক নয়।

প্রথম ক্যেত্রে `age` property নেই।

দ্বিতীয় ক্যেত্রে `age` property আছে, কিন্ত তার value `undefined`।

এই difference property existence checking-এর সময় important।

26. Nested Object

Object-এর ভিতরে আরেকটি object রাখা যায়।

```
const user = {  
  name: "Shariar",  
  age: 31,  
  address: {  
    city: "Dhaka",  
    country: "Bangladesh"  
  }  
};
```

এখানে:

```
user  
├──  
├── name  
├── age  
└── address  
    ├──  
    ├── city  
    └── country
```

27. Nested Property Access

প্রথমে:

```
user.address
```

তারপর:

```
user.address.city
```

Example:

```
console.log(user.address.city);
```

Output:

```
Dhaka
```

28. Deeply Nested Object

একাধিক level থাকতে পারে।

```
const company = {  
  name: "Tech Company",  
  office: {  
    location: {  
      city: "Dhaka",  
      country: {  
        name: "Bangladesh"  
      }  
    }  
  }  
};
```

Access:

```
console.log(company.office.location.city);
```

Output:

```
Dhaka
```

29. Nested Object Update

Nested property-ও update করা যায়।

```
const user = {  
  name: "Shariar",  
  address: {  
    city: "Dhaka"  
  }  
};  
  
user.address.city = "Rangpur";
```

এখন:

```
console.log(user.address.city);
```

Output:

```
Rangpur
```

30. Object-এর মধ্যে Array

Object-এর property হিসেবে array রাখা খুব common।

```
const user = {  
  name: "Shariar",  
  skills: ["JavaScript", "React", "Next.js"]  
};
```

Array access:

```
console.log(user.skills[0]);
```

Output:

```
JavaScript
```

31. Array-এর মধ্যে Object

উন্টোটাও খুব common।

```
const users = [  
  {  
    name: "Shariar",  
    age: 31  
  },  
  {  
    name: "Rahim",  
    age: 25  
  }  
];
```

Access:

```
console.log(users[0].name);
```

Output:

```
Shariar
```

এটা API response এবং React development-এ অত্যন্ত common pattern।

32. `typeof` Object

Object-এর type check করলে:

```
const user = {  
  name: "Shariar"  
};  
  
console.log(typeof user);
```

Output:

```
object
```

33. Array-এর `typeof` কেন Object?

```
console.log(typeof []);
```

Output:

```
object
```

কারণ JavaScript-এর array technically object-এর একটি specialized form।

তাই array check করতে:

```
Array.isArray(value);
```

ব্যবহার করা উচিত।

34. Object Reference Basics

এটা JavaScript Object-এর খুব important concept।

দুটো variable একই object-কে reference করতে পারে।

```
const user1 = {  
  name: "Shariar"  
};  
  
const user2 = user1;
```

এখানে নতুন object তৈরি হয়নি।

দুটো variable একই object-কে refer করছে।

Conceptually:

```
user1 ———┐  
           ↓  
       { name: "Shariar" }  
           ↑  
user2 ———┐
```

35. Reference-এর কারণে কী হয়?

```
const user1 = {  
  name: "Shariar"  
};  
  
const user2 = user1;  
  
user2.name = "Duke";  
  
console.log(user1.name);
```

Output:

```
Duke
```

কারণ `user1` এবং `user2` একই object-এর reference ধরে আছে।

36. Object Compare করলে কী হয়?

দুইটা দেখতে একই object হলেও:

```
const user1 = {  
  name: "Shariar"  
};  
  
const user2 = {  
  name: "Shariar"  
};  
  
console.log(user1 === user2);
```

Output:

```
false
```

কারণ দুইটা আলাদা object।

37. Same Reference হলে

```
const user1 = {  
  name: "Shariar"  
};  
  
const user2 = user1;  
  
console.log(user1 === user2);
```

Output:

```
true
```

কারণ দুটো একই object reference করছে।

38. Object Property Existence

কোনো property object-এর মধ্যে আছে কিনা check করার কয়েকটি উপায় আছে।

সবচেয়ে গুরুত্বপূর্ণ:

```
in
```

```
hasOwnProperty()
```

```
Object.hasOwnProperty()
```

39. **in** Operator

Syntax:

```
"property" in object
```

Example:

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
console.log("name" in user);  
console.log("email" in user);
```

Output:

```
true  
false
```

40. `in` কী Check করে?

`in` object-এর property chain-এর মধ্যে property আছে কিনা check করে।
এতে inherited property-ও ধরা পড়তে পারে।

Example:

```
const user = {  
  name: "Shariar"  
};  
  
console.log("name" in user);
```

Output:

```
true
```

41. `hasOwnProperty()`

এটি check করে property-টি নিজের **property** কিনা।

```
const user = {  
  name: "Shariar"  
};  
  
console.log(user.hasOwnProperty("name"));  
console.log(user.hasOwnProperty("age"));
```

Output:

```
true  
false
```

42. `Object.hasOwnProperty()`

Modern JavaScript-এ:

```
Object.hasOwnProperty(object, property)
```

ব্যবহার করা যায়।

Example:

```
const user = {  
  name: "Shariar"  
};  
  
console.log(Object.hasOwnProperty(user, "name"));  
console.log(Object.hasOwnProperty(user, "age"));
```

Output:

```
true  
false
```

এটি নিজের property check করার clean এবং recommended approach।

43. `in` বনাম `Object.hasOwn()`

Feature	<code>in</code>	<code>Object.hasOwn()</code>
Own property check	✗	✓
Inherited property check	✓	✗
Syntax	<code>"key" in obj</code>	<code>Object.hasOwn(obj, "key")</code>
Common use	Property exists anywhere in prototype chain	নিজের property check

সাধারণ application code-এ own property check করতে:

```
Object.hasOwn(obj, key);
```

খুব useful।

44. কেন `hasOwnProperty()` সবসময় safest নয়?

সাধারণ object-এ:

```
obj.hasOwnProperty("name");
```

কাজ করে।

কিন্তু object-এর নিজের property হিসেবে যদি `hasOwnProperty` overwrite করা থাকে:

```
const obj = {  
  hasOwnProperty: "hello"  
};
```

তাহলে:

```
obj.hasOwnProperty("name");
```

call করা যাবে না।

তাই modern code-এ:

```
Object.hasOwn(obj, "name");
```

আরও robust।

45. Property Access — Practical Example

ধরো API থেকে user data এসেছে:

```
const user = {
  id: 101,
  name: "Shariar",
  email: "shariar@example.com",
  role: "developer",
  address: {
    city: "Dhaka"
  }
};
```

Data access:

```
console.log(user.name);
console.log(user.email);
console.log(user.address.city);
```

46. Dynamic Property Access — Practical Example

ধরো form-এর field dynamically update করতে হবে।

```
const user = {
  name: "Shariar",
  email: "shariar@example.com"
};

const field = "email";

console.log(user[field]);
```

Output:

```
shariar@example.com
```

এই pattern React form এবং dynamic UI-তে খুব important।

47. Object Property Key হিসেবে String

Object-এর property key সাধারণত string হিসেবে কাজ করে।

```
const user = {  
  name: "Shariar"  
};
```

এটা internally property key `"name"` হিসেবে কাজ করে।

Number key দিনে:

```
const obj = {  
  1: "one"  
};  
  
console.log(obj[1]);  
console.log(obj["1"]);
```

দুটোই একই property access করতে পারে।

48. Object Property Key হিসেবে Variable

Object literal-এর মধ্যে:

```
const key = "name";  
  
const user = {  
  [key]: "Shariar"  
};
```

এখানে `[key]` হলো **computed property name**।

এটা পরের Object doc-এ বিস্তারিতভাবে আসবে।

এখন শুধু বুঝে রাখো:

```
[key]
```

মানে variable-এর value-কে property name হিসেবে ব্যবহার করা।

49. Object-এ Function

Object-এর property হিসেবে function রাখা যায়।

```
const user = {  
  name: "Shariar",  
  
  greet() {  
    console.log("Hello!");  
  }  
};
```

Call:

```
user.greet();
```

Output:

```
Hello!
```

Object-এর ভিতরের function-কে সাধারণভাবে **method** বলা হয়।

এটা পরবর্তী object concepts-এ আরও বিস্তারিত আসবে।

50. Object এবং `const`

অনেক beginner ভাবে:

`const` object হলে object change করা যাবে না।

এটা ভুল।

```
const user = {  
  name: "Shariar"  
};  
  
user.name = "Duke";
```

এটা valid।

কারণ `const` variable-টাকে অন্য object-এ reassign করতে বাধা দেয়।

```
user = {};
```

এটা invalid।

কিন্তু object-এর property পরিবর্তন করা যায়।

51. `const` Object Mental Model

```
const user = {  
  name: "Shariar"  
};
```

`const` মূলত reference binding-কে protect করছে।

```
user —————> { name: "Shariar" }
```

তুমি object-এর ভিতরের data change করতে পারো:

```
user.name = "Duke";
```

কিন্তু:

```
user = anotherObject;
```

করতে পারবে না।

52. Object Creation — `new Object()`

আরেকভাবে object তৈরি করা যায়:

```
const user = new Object();
```

তারপর:

```
user.name = "Shariar";  
user.age = 31;
```

Result:

```
{  
  name: "Shariar",  
  age: 31  
}
```

53. `{}` বনাম `new Object()`

দুটো দিয়েই object তৈরি করা সম্ভব।

```
const user1 = {};
```

```
const user2 = new Object();
```

কিন্তু modern JavaScript-এ সাধারণত:

```
{}
```

ব্যবহার করা হয়।

কারণ এটা:

- shorter
- cleaner
- easier to read
- বেশি common

54. Object Creation-এর Recommended Way

সাধারণ object-এর জন্ম:

```
const user = {  
  name: "Shariar",  
  age: 31  
};
```

এটাই preferred।

`new Object()` জানা দরকার কারণ JavaScript-এর syntax এবং legacy code বুঝতে কাজে লাগবে, কিন্তু সাধারণ application code-এ এর প্রয়োজন খুব কম।

55. Object-এর Property Update Flow

একটা simple CRUD mental model:

```
CREATE  
  ↓  
READ  
  ↓  
UPDATE  
  ↓  
DELETE
```

Example:

Create

```
const user = {};
```

Add

```
user.name = "Shariar";
```

Read

```
console.log(user.name);
```

Update

```
user.name = "Duke";
```

Delete

```
delete user.name;
```

56. Real-World Example — Product

```
const product = {  
  id: 101,  
  name: "Laptop",  
  price: 80000,  
  stock: 5  
};
```

Read:

```
console.log(product.name);
```

Update:

```
product.price = 75000;
```

Add:

```
product.brand = "HP";
```

Delete:

```
delete product.stock;
```

57. Real-World Example — User

```
const user = {  
  id: 1,  
  name: "Shariar",  
  email: "shariar@example.com",  
  isActive: true  
};
```

Check:

```
console.log(user.name);
```

Update:

```
user.isActive = false;
```

Add:

```
user.role = "developer";
```

Delete:

```
delete user.role;
```

58. Common Mistake — Dot Notation with Dynamic Key

ভুল:

```
const key = "name";  
  
console.log(user.key);
```

এটা `"key"` property খুঁজবে।

সঠিক:

```
console.log(user[key]);
```

59. Common Mistake — Missing Property

```
console.log(user.email);
```

যদি `email` না থাকে:

```
undefined
```

তাই nested object access করার সময় সাবধান হতে হবে।

```
console.log(user.address.city);
```

যদি `address` না থাকে, error হতে পারে:

```
Cannot read properties of undefined
```

এ ধরনের problem handle করতে পরে **optional chaining** (`?.`) খুব useful হবে।

60. Common Mistake — Object Comparison

ভুল ধারণা:

```
{ name: "Shariar" } === { name: "Shariar" }
```

Result:

```
false
```

কারণ object content দেখে equality হয় না; reference compare হয়।

61. Common Mistake — `const` মানেই

Immutable

```
const user = {  
  name: "Shariar"  
};  
  
user.name = "Duke";
```

এটা valid।

`const` object-এর property mutation বন্ধ করে না।

62. Common Mistake — `typeof null`

একটা famous JavaScript behavior:

```
console.log(typeof null);
```

Output:

```
object
```

এটা JavaScript-এর historical legacy behavior।

তাই:

```
typeof value === "object"
```

দিয়ে কোনো value সত্যিই normal object কিনা নিশ্চিত হওয়া যায় না।

63. Object Fundamentals — Quick Comparison

Task	Syntax
Create	<code>const user = {}</code>
Read	<code>user.name</code>
Dynamic read	<code>user[key]</code>
Add	<code>user.age = 31</code>
Update	<code>user.age = 32</code>
Delete	<code>delete user.age</code>
Check exists	<code>"age" in user</code>
Check own property	<code>Object.hasOwn(user, "age")</code>
Nested access	<code>user.address.city</code>

64. Dot vs Bracket — Quick Table

Situation	Best choice
Normal property	<code>user.name</code>
Dynamic property	<code>user[key]</code>
Property contains space	<code>user["full name"]</code>
Property contains <code>-</code>	<code>user["first-name"]</code>
Variable-based key	<code>user[key]</code>

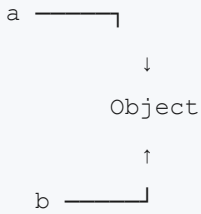
65. Object Reference — Quick Mental Model

মনে রাখো:

```
const a = {
  value: 10
};

const b = a;
```

এখানে:



b নতুন object না।

তাই:

```
b.value = 20;
```

করলে:

```
console.log(a.value);
```

Output:

20

66. Object Fundamentals — Practice Questions

Question 1

Output কী হবে?

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
console.log(user.name);
```

Question 2

Output कै?

```
const user = {  
  name: "Shariar"  
};  
  
const key = "name";  
  
console.log(user[key]);
```

Question 3

Output कै?

```
const user = {  
  name: "Shariar"  
};  
  
const key = "name";  
  
console.log(user.key);
```

Question 4

Output कै?

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
user.age = 32;  
  
console.log(user.age);
```

Question 5

Output কী?

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
delete user.age;  
  
console.log(user.age);
```

Question 6

Output কী?

```
const user = {  
  address: {  
    city: "Dhaka"  
  }  
};  
  
console.log(user.address.city);
```

Question 7

Output কী?

```
const user1 = {  
  name: "Shariar"  
};  
  
const user2 = user1;  
  
user2.name = "Duke";  
  
console.log(user1.name);
```

Question 8

Output কী?

```
const user1 = {  
  name: "Shariar"  
};  
  
const user2 = {  
  name: "Shariar"  
};  
  
console.log(user1 === user2);
```

67. Coding Challenges

Challenge 1 — User Object

একটা object তৈরি করো:

```
name  
age  
email  
isActive
```

তারপর:

1. name print করো
 2. age update করো
 3. নতুন `role` property add করো
 4. email delete করো
-

Challenge 2 — Dynamic Access

```
const user = {  
  name: "Shariar",  
  email: "shariar@example.com",  
  age: 31  
};
```

একটা variable:

```
const key = "email";
```

ব্যবহার করে email access করো।

Challenge 3 — Nested Object

এই structure তৈরি করো:

```
user  
├── name  
├── age  
└── address  
    ├── city  
    └── country
```

তারপর `city` এবং `country` print করো।

Challenge 4 — Property Check

```
const product = {  
  name: "Laptop",  
  price: 80000  
};
```

Check করো:

```
name আছে?  
stock আছে?  
price আছে?
```

`Object.hasOwnProperty()` ব্যবহার করে।

68. Interview Questions

1. JavaScript Object কী?

Answer:

Object হলো key-value pair ভিত্তিক data structure, যা related data এবং behavior organize করতে ব্যবহৃত হয়।

2. Dot notation এবং bracket notation-এর difference কী?

```
user.name;
```

এবং:

```
user["name"];
```

দুটো property access করে।

কিন্তু dynamic key-এর ক্ষেত্রে bracket notation প্রয়োজন:

```
const key = "name";  
  
user[key];
```

3. `user.key` এবং `user[key]` কি একই?

না।

```
user.key
```

`"key"` property access করে।

```
user[key]
```

`key` variable-এর value ব্যবহার করে property access করে।

4. Object property কীভাবে add করা হয়?

```
user.age = 31;
```

অথবা:

```
user["age"] = 31;
```

5. Object property কীভাবে delete করা হয়?

```
delete user.age;
```

6. দুইটা identical object `===` করলে false কেন?

কারণ object equality reference-based।

```
{ } === { }
```

ফলাফল:

```
false
```

7. `const` object-এর property পরিবর্তন করা যায়?

হ্যাঁ।

```
const user = {  
  name: "Shariar"  
};  
  
user.name = "Duke";
```

Valid।

কিন্তু:

```
user = {};
```

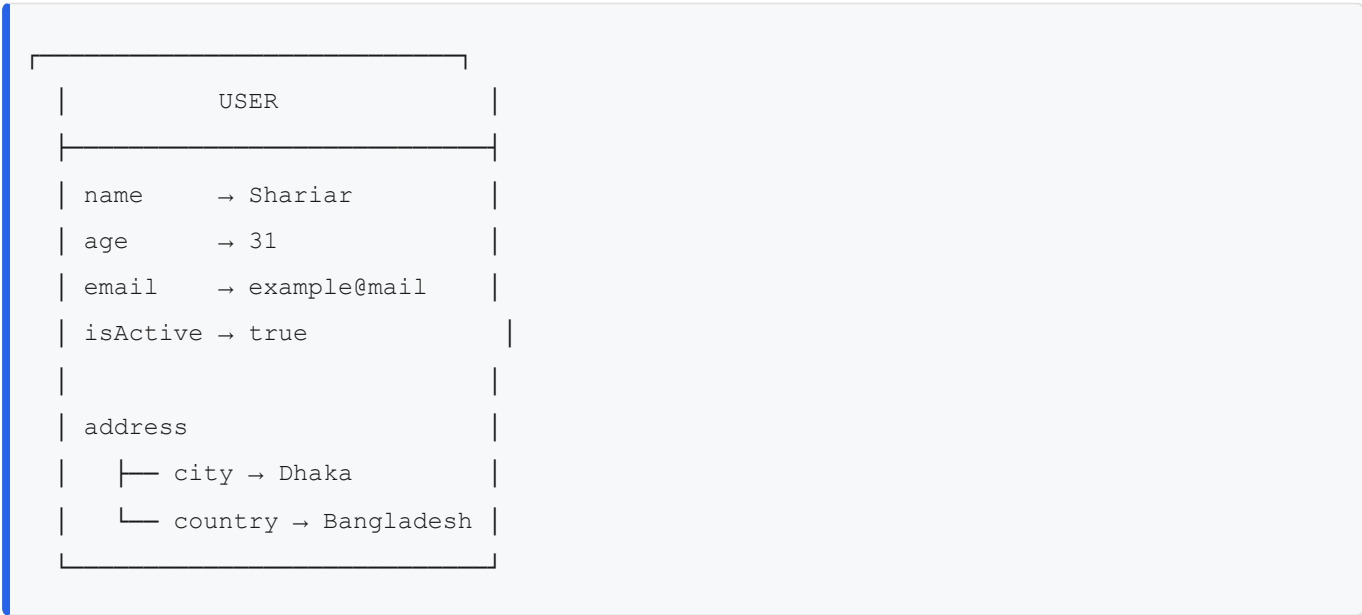
Invalid!

8. `in` এবং `Object.hasOwnProperty()` -এর difference কী?

`in` prototype chain-সহ property খোঁজে। `Object.hasOwnProperty()` শুধু object's own property check করে।

69. Mental Model

Object-কে একটা **real-world profile card** হিসেবে চিন্তা করো।



তখন:

```
user.name
```

মানে:

User-এর name দাও।

```
user.address.city
```

মানে:

User → address → city-তে যাও।

```
user[key]
```

মানে:

key variable যেই property দেখাচ্ছে, সেখানে যাও।

```
delete user.age
```

মানে:

User card থেকে age information remove করো।

70. Quick Revision

Object

- |
 - ├─ Create
 - | └─ {}
 - | └─ new Object()
 - ├─ Access
 - | └─ dot notation
 - | └─ bracket notation
 - ├─ Modify
 - | └─ add
 - | └─ update
 - | └─ delete
 - ├─ Structure
 - | └─ nested object
 - | └─ object inside array
 - | └─ array inside object
 - ├─ Reference
 - | └─ same reference
 - | └─ reference comparison
 - └─ Property existence
 - | └─ in
 - | └─ hasOwnProperty()
 - | └─ Object.hasOwnProperty()

71. Golden Rules

🏆 Rule 1 Object = **key-value pairs**!

🏆 Rule 2 Normal property access-এর জন্য **dot notation** cleanest!

🏆 Rule 3 Dynamic key হলে **bracket notation** ব্যবহার করো।

🏆 Rule 4 Property না থাকলে সাধারণত `undefined` পাওয়া যায়।

🏆 Rule 5 Object-এর property add/update করতে assignment ব্যবহার করা যায়।

🏆 Rule 6 Property remove করতে `delete` ব্যবহার করা হয়।

🏆 Rule 7 Object comparison content-based নয়, reference-based।

🏆 Rule 8 `const` object-কে reassign করা যায় না, কিন্তু property পরিবর্তন করা যায়।

🏆 Rule 9 Own property check করার জন্য `Object.hasOwnProperty()` মনে রাখো।

🏆 Rule 10 Nested object মানে object-এর ভিতরে object—access করতে multiple property access ব্যবহার করা হয়।

72. Final Mastery Checklist

এই Doc শেষ করার পর তোমার এগুলো confidently পারা উচিত:

- ☐ Object কী explain করতে পারি
- ☐ Object literal দিয়ে object বানাতে পারি
- ☐ `new Object()` চিনতে পারি
- ☐ Empty object তৈরি করতে পারি
- ☐ Property এবং value explain করতে পারি
- ☐ Dot notation ব্যবহার করতে পারি
- ☐ Bracket notation ব্যবহার করতে পারি
- ☐ Dynamic property access করতে পারি
- ☐ Property add করতে পারি
- ☐ Property update করতে পারি
- ☐ Property delete করতে পারি
- ☐ Missing property বুঝতে পারি
- ☐ Nested object access করতে পারি
- ☐ Object-এর মধ্যে array ব্যবহার করতে পারি
- ☐ Array-এর মধ্যে object ব্যবহার করতে পারি
- ☐ Object reference বুঝতে পারি
- ☐ Object equality কেন reference-based বুঝতে পারি
- ☐ `in` operator বুঝতে পারি
- ☐ `hasOwnProperty()` বুঝতে পারি

- ☐ `Object.hasOwnProperty()` বুঝতে পারি
 - ☐ `const` object-এর behavior বুঝতে পারি
 - ☐ Object fundamentals নিয়ে basic interview question answer করতে পারি
-

Next Step

এরপর আসবে:

Doc 7.2 — Object Static Methods & Object Copying

যেখানে থাকবে:

```
Object.keys()
Object.values()
Object.entries()
Object.assign()
Object spread (...)
Object copy
Object merge
Shallow copy
Nested reference problem
Immutable object update
React state object update
```

এগুলো Object-এর খুবই গুরুত্বপূর্ণ practical concepts—বিশেষ করে React/Next.js development-এর জন্য।